

CLAUDE CODE FIELD GUIDE · KOREAN EDITION

Claude Code

실전 완전정복

설정팩과 함께 배우는 AI 코딩 에이전트 실무 활용 가이드
CLAUDE.md · 커스텀 커맨드 · 서브에이전트 · 훅 · MCP

커맨드 35종

에이전트 11종

혹 15종 동봉

```
$ npm install -g @anthropic-ai/claude-code
```

이 책을 읽어야 하는 이유

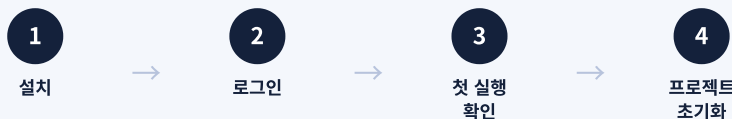
Claude Code를 "설치해서 채팅하듯 쓰는 것"과 "제대로 세팅해서 쓰는 것"의 생산성 차이는 체감상 3~5배 이상입니다. 하지만 공식 문서는 영어이고, 한국어 자료 대부분은 설치법 수준에서 멈춰 있습니다. CLAUDE.md 설계, 커스텀 커맨드, 서브에이전트, 훅(Hooks), MCP 연동까지 다루는 한국어 실전 자료는 아직 드뭅니다.

이 책은 다음을 약속합니다.

- 모든 예제는 **실제로 작동하는 설정 파일**입니다. 이 책과 함께 제공되는 설정팩 (claude-forge)에는 커스텀 커맨드 35종, 서브에이전트 11종, 자동화 훅 15종, 규칙 파일 8종이 포함되어 있고, 책의 예제는 전부 이 설정팩에서 발췌했습니다.
- "이런 게 가능하다" 수준이 아니라 **파일 경로, JSON 키, 스크립트 내용**까지 그대로 따라 할 수 있게 씁니다.
- 실무 시나리오(새 기능 개발, 버그 수정, 보안 감사, 리팩토링) 기준으로 워크플로우를 구성합니다.

대상 독자: Claude Code를 써봤지만 "그냥 좋은 채팅" 수준으로 쓰고 있는 개발자, 바이트코딩을 시작했는데 프로젝트가 커지면서 한계를 느끼는 분, 팀에 AI 코딩 도구를 도입하려는 리더.

1장. 설치와 첫 실행 — 5분 안에 끝내기



Claude Code는 터미널에서 동작하는 AI 코딩 에이전트입니다. 채팅창에 코드를 붙여넣는 방식과 달리, 에이전트가 직접 파일을 읽고, 수정하고, 테스트를 돌리고, 커밋합니다.

실행 절차

1. **설치:** 터미널에서 한 줄이면 됩니다.

```
npm install -g @anthropic-ai/claude-code
```

macOS는 Homebrew(`brew install claude-code`), Windows는 WSL 환경을 권장합니다. 네이티브 Windows도 지원되지만 심볼릭 링크 기반 설정 공유가 제한됩니다.

2. **로그인:** 프로젝트 폴더에서 `claude` 를 실행하면 브라우저 인증이 열립니다. Claude Pro/Max 구독 또는 API 키 어느 쪽으로도 사용할 수 있습니다.
3. **첫 실행 확인:** 아무 프로젝트 폴더에서 다음을 시험해보세요.

4. **프로젝트 초기화:** `/init` 명령을 실행하면 Claude가 코드베이스를 분석해 CLAUDE.md 초안을 만들어줍니다. 이 파일이 무엇인지는 2장에서 다룹니다.

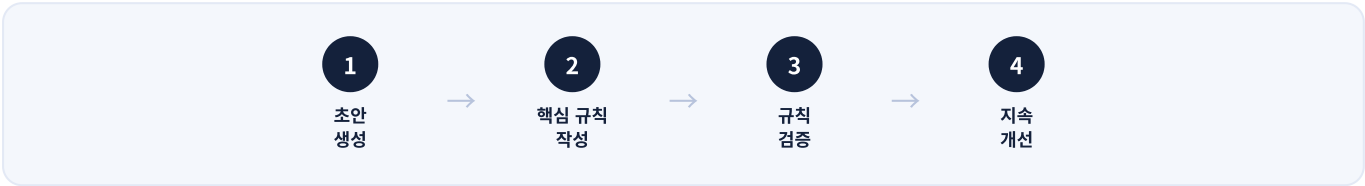
알아두면 좋은 실행 모드

모드	명령	용도
대화형	<code>claude</code>	기본. 터미널에서 대화하며 작업
헤드리스	<code>claude -p "작업 내용"</code>	스크립트/CI에서 일회성 실행
계속하기	<code>claude -c</code>	직전 세션 이어서
플랜 모드	세션 중 Shift+Tab	수정 없이 계획만 세우게 하기

팁: 처음 며칠은 플랜 모드를 적극 쓰세요. Claude가 뭘 하려는지 계획을 먼저 보고

승인하는 습관이 들면, 나중에 자율 실행을 시켜도 결과를 예측할 수 있게 됩니다.

2장. CLAUDE.md — 프로젝트의 기억을 설계한다



Claude Code는 세션을 시작할 때 프로젝트 루트의 `CLAUDE.md` 파일을 자동으로 읽습니다. 여기에 적힌 내용은 매 대화에 항상 적용되는 "프로젝트 헌법"이 됩니다. 이 파일의 품질이 곧 결과물의 품질입니다.

실행 절차

1. **생성:** `/init` 으로 초안을 만들거나 직접 작성합니다.
2. **핵심 규칙 작성:** 아래 항목은 반드시 포함하세요.

```
# 프로젝트: 주문 관리 API
```

```
## 기술 스택
```

```
- TypeScript 5 + Node.js 22, Express, Prisma, PostgreSQL
```

```
## 명령어
```

```
- 테스트: npm test (vitest)  
- 린트: npm run lint  
- 빌드: npm run build
```

```
## 코딩 규칙
```

```
- 모든 금액 계산은 정수(원 단위)로. 부동소수점 금지  
- API 응답은 반드시 responses.ts의 헬퍼로 감쌀 것  
- DB 마이그레이션 파일은 직접 수정 금지
```

```
## 하지 말 것
```

```
- .env 파일 읽거나 수정하지 말 것  
- main 브랜치에 직접 커밋하지 말 것
```

3. **검증**: 새 세션을 열고 규칙에 어긋나는 요청을 일부러 해보세요. ("금액을 float로 계산해줘") Claude가 규칙을 근거로 거절하거나 대안을 제시하면 성공입니다.

4. **지속 개선**: Claude가 같은 실수를 반복하면 그 자리에서 `#` 키를 눌러 규칙을 추가하세요. CLAUDE.md는 한 번 쓰고 끝나는 문서가 아니라 실수가 누적될수록 강해지는 문서입니다.

잘 쓴 CLAUDE.md의 3원칙

- **짧게**: 모든 세션의 컨텍스트를 차지하므로, 장황한 설명보다 명령형 규칙 위주로.
- **구체적으로**: "코드를 깔끔하게"(나쁨) → "함수는 30줄 이내, 매개변수 4개 이하"(좋음)
- **금지 목록 포함**: 해야 할 것보다 하지 말아야 할 것이 사고를 더 잘 막습니다.

설정팩의 `rules/` 폴더에는 코딩 스타일, git 워크플로우, 테스트, 보안 등 8개 규칙 파일이 분리되어 있습니다. 규모가 커지면 이렇게 주제별로 파일을 나누고 CLAUDE.md에서 참조하는 구조가 유지보수에 유리합니다.

3장. settings.json — 권한과 환경을 통제한다

`.claude/settings.json` (프로젝트) 또는 `~/.claude/settings.json` (전역)으로 Claude Code의 동작을 세밀하게 제어할 수 있습니다. 특히 **권한(permissions)** 설정은 안전과 생산성을 동시에 잡는 핵심입니다.

자주 쓰는 설정 예시

```
{
  "permissions": {
    "allow": [
      "Bash(npm test:*)",
      "Bash(npm run lint:*)",
      "Bash(git status)",
      "Bash(git diff:*)",
      "Read(**)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Read(.env*)",
      "Read(**/secrets/**)"
    ]
  },
  "env": {
    "NODE_ENV": "development"
  }
}
```

이렇게 하면 테스트/린트/조회성 git 명령은 매번 승인 버튼을 누르지 않아도 자동 실행되고, 위험한 명령과 시크릿 파일 접근은 아예 차단됩니다.

권한 설계의 사고방식

1. 처음엔 기본값(매번 확인)으로 시작합니다.
2. 반복해서 승인하게 되는 안전한 명령을 `allow`에 추가합니다.
3. 절대 일어나면 안 되는 일을 `deny`에 추가합니다. `deny`가 `allow`보다 우선합니다.
4. 팀 프로젝트라면 `.claude/settings.json`을 git에 커밋해 팀 전체가 같은 가드레일을 공유하고, 개인 취향은 `.claude/settings.local.json` (git 제외)에 둡니다.

경고: `--dangerously-skip-permissions` 플래그는 모든 확인을 건너뛵니다. 격리된

컨테이너가 아니라면 쓰지 마세요. 편하다고 로컬에서 커두는 순간, "rm -rf 사고"의

주인공이 될 수 있습니다.

4장. 커스텀 슬래시 커맨드 — 반복 작업을 명령어로 만든다



`.claude/commands/` 폴더에 마크다운 파일을 넣으면 그 파일명이 곧 슬래시 커맨드가 됩니다. "매번 길게 설명하던 반복 요청"을 한 단어로 압축하는 기능입니다.

실행 절차

- 1. **파일 생성:** `.claude/commands/quick-commit.md` 를 만듭니다.
- 2. **프롬프트 작성:** 파일 내용이 곧 실행될 지시문입니다.

```
---
description: 변경사항을 분석해 컨벤션에 맞는 커밋 메시지로 커밋
allowed-tools: Bash(git:*), Read
---

현재 스테이징된 변경사항을 분석하고:
1. conventional commits 형식(feat/fix/refactor/docs)으로
   커밋 메시지를 작성하라
2. 제목은 50자 이내, 본문에는 "왜"를 적어라
3. 커밋을 실행하라
```

- 3. **인자 처리:** 지시문 안에 `$ARGUMENTS` 를 쓰면 `/quick-commit 긴급 수정` 처럼 호출 시 넘긴 인자가 그 자리에 치환됩니다.
- 4. **테스트:** 새 세션에서 `/quick-commit` 을 실행해 의도대로 동작하는지 확인합니다.

설정팩에 포함된 커맨드 중 활용도 높은 것들

커맨드	하는 일
<code>/plan</code>	구현 전 계획 수립을 강제. 큰 작업 전 필수
<code>/tdd</code>	테스트 먼저(RED) → 최소 구현(GREEN) → 리팩토링 사이클 강제
<code>/code-review</code>	심각도(CRITICAL~LOW) 등급을 매기는 체계적 코드 리뷰
<code>/security-review</code>	OWASP 기준 보안 점검
<code>/verify-loop</code>	빌드·테스트·린트가 모두 통과할 때까지 자동 반복 수정
<code>/checkpoint</code>	현재 작업 상태 저장 (긴 작업 중간 저장용)
<code>/worktree-start</code>	git worktree로 병렬 작업 브랜치 생성

핵심 통찰은 이것입니다: **좋은 커맨드는 "요청"이 아니라 "절차"를 담습니다.** "코드 리뷰해줘"가 아니라, "1단계: 스펙 준수 확인 → 2단계: 보안 점검 → 3단계: 심각도 등급 부여 → 4단계: 파일:줄번호와 수정안 제시"처럼 절차를 명시할수록 결과가 일관됩니다.

5장. 서버에이전트 — 역할별 전문가를 고용한다



`.claude/agents/` 폴더의 마크다운 파일은 각각 독립된 컨텍스트에서 동작하는 서버에이전트가 됩니다. 메인 세션의 컨텍스트를 오염시키지 않고 전문 작업을 위임할 수 있습니다.

실행 절차

1. **역할 정의**: 에이전트 하나 = 역할 하나. "다 잘하는 에이전트"는 아무것도 잘하지 못합니다.
2. **도구 제한**: 리뷰어에게 Write 권한을 주지 마세요. 읽기 전용 에이전트는 사고를 칠 수 없습니다.
3. **모델 선택**: 깊은 추론이 필요한 역할(아키텍트, 리뷰어)은 상위 모델, 기계적 실행(빌드 에러 수정, 문서 갱신)은 빠른 모델로.
4. **호출**: 이름을 불러 명시적으로("code-reviewer로 이 PR 검토해줘") 또는 description에 조건을 적어 자동 위임되게 합니다.

실제 에이전트 정의 예시 (설정팩 `agents/code-reviewer.md` 발췌)

```
---
name: code-reviewer
description: 코드 품질·보안 전문 리뷰어. 코드 수정 후 반드시 사용.
tools: ["Read", "Grep", "Glob", "Bash"]
model: opus
---
```

너는 코드 리뷰어다. 임무: 체계적이고 심각도 기반의 리뷰.

성공 기준:

- 스펙 준수를 코드 품질보다 먼저 검증한다
- 모든 지적은 파일:줄번호를 명시한다
- 심각도 등급: CRITICAL / HIGH / MEDIUM / LOW
- 각 지적마다 구체적 수정안을 제시한다
- 결론은 셋 중 하나: APPROVE / REQUEST CHANGES / COMMENT

주목할 부분: `tools` 에 Write가 없습니다. 이 리뷰어는 지적만 할 수 있고 고치지는 못합니다. 수정은 메인 세션 (또는 실행 담당 에이전트)의 몫으로 남겨서, "리뷰하다가 멋대로 코드를 바꿔버리는" 사고를 구조적으로 차단합니다.

설정팩의 에이전트 구성 (11종)

- 설계·계획: `architect`, `planner`
- 품질: `code-reviewer`, `security-reviewer`, `database-reviewer`, `tdd-guide`
- 실행·수리: `build-error-resolver`, `refactor-cleaner`, `e2e-runner`
- 유지보수: `doc-updater`, `verify-agent`

6장. 훅(Hooks) — 사람이 안 지켜봐도 되는 가드레일



훅은 Claude Code의 특정 이벤트(도구 실행 전/후, 세션 시작, 프롬프트 제출 등)에 자동으로 실행되는 셸 스크립트입니다. "Claude가 알아서 잘하겠지"를 "구조적으로 사고가 불가능하다"로 바꾸는 장치입니다.

실행 절차

1. 이벤트 선택: 언제 개입할지 정합니다.

이벤트	시점	대표 용도
PreToolUse	도구 실행 직전	위험 명령 차단
PostToolUse	도구 실행 직후	결과 검사, 시크릿 필터링
UserPromptSubmit	프롬프트 제출 시	컨텍스트 자동 주입
SessionStart	세션 시작 시	상태 복원, 리마인더
Stop	응답 완료 시	작업 로그 기록

2. 스크립트 작성: 예 — 원격 배포 명령을 차단하는 가드.

```
#!/bin/bash
# remote-command-guard.sh (PreToolUse, matcher: Bash)
input=$(cat)
cmd=$(echo "$input" | jq -r '.tool_input.command')

if echo "$cmd" | grep -qE "(kubectl|terraform apply|aws .* delete)"; then
  echo '{"decision": "block", "reason": "원격 인프라 명령은 수동 실행 필요"}'
  exit 0
fi
exit 0
```

3. 등록: settings.json의 hooks 섹션에 연결합니다.

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command",
        "command": "~/claude/hooks/remote-command-guard.sh" }]
    }]
  }
}
```

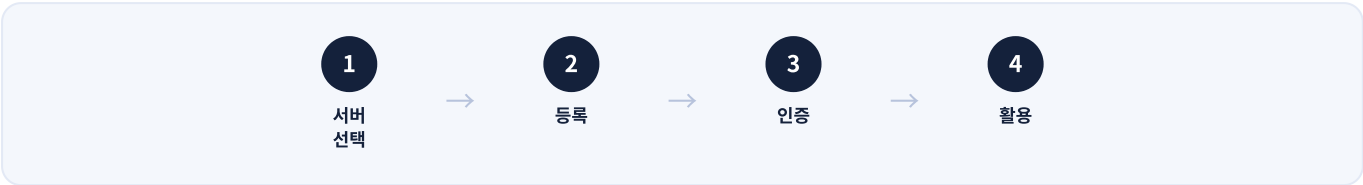
4. **테스트**: 차단 대상 명령을 일부러 요청해서 혹은 막는지 확인합니다.

설정팩에 포함된 혹은 핵심 3종

- `output-secret-filter.sh` (PostToolUse): 도구 출력에서 API 키·토큰 패턴을 감지해 마스킹. 시크릿이 대화 로그에 남는 사고 방지.
- `db-guard.sh`: DROP/TRUNCATE 등 파괴적 DB 명령 차단.
- `rate-limiter.sh` (mcp__* matcher): MCP 도구 호출 빈도를 제한해 외부 API 요금 폭탄 방지.

혹과 권한 설정(3장)의 차이: 권한은 "무엇을 할 수 있나"의 정적 목록이고, 혹은 명령 내용을 스크립트로 검사하는 동적 로직입니다. 정규식·조건문이 필요한 규칙은 혹은으로, 단순 목록은 권한으로 처리하세요.

7장. MCP 서버 — Claude의 손을 외부 세계로 확장한다



MCP(Model Context Protocol)는 Claude Code에 외부 도구를 연결하는 표준입니다. DB 조회, 라이브러리 최신 문서, 웹 검색, GitHub 조작 등을 대화 안에서 직접 수행하게 됩니다.

실행 절차

- 서버 선택**: 필요한 것만 답니다. MCP 서버 하나하나가 컨텍스트와 시작 시간을 잡아먹으므로 "일단 다 설치"는 하수입니다.
- 등록**: CLI로 등록하거나 `.mcp.json` 을 프로젝트에 커밋합니다.

```
claude mcp add context7 -- npx -y @upstash/context7-mcp@latest
```

- 인증**: GitHub 등 인증이 필요한 서버는 환경변수로 토큰을 주입합니다. 토큰을 설정 파일에 하드코딩하지 말고 `${GITHUB_TOKEN}` 참조를 쓰세요.
- 활용**: 등록 후에는 자연어로 그냥 요청하면 됩니다. ("React 19의 use() 혹은 최신 문서 확인해서 이 코드 고쳐 줘")

검증된 범용 조합 (설정팩 기본 구성)

서버	용도	체감 효과
context7	라이브러리 최신 문서 주입	구버전 API로 코드 짜는 문제 해결
memory	세션 간 지식 그래프 유지	"지난번에 말했잖아"가 통하게 됨
github	PR/이슈/리포 조작	리뷰→수정→PR 사이클을 대화로
exa	AI 웹 검색	에러 메시지 최신 해결책 검색

비용 주의: MCP 도구 호출이 외부 유료 API를 쓰는 경우가 있습니다. 설정팩의

`expensive-mcp-warning.sh` 혹처럼 고비용 도구 호출 전 경고를 띄우는 안전장치를

권장합니다.

8장. 실전 워크플로우 4종 — 조합이 실력이다

WORKFLOW 1

새 기능 개발

`/plan` → `/tdd` → `/code-review`

WORKFLOW 2

버그 수정

재현 테스트 → 최소 수정 →
`/verify-loop`

WORKFLOW 3

보안 감사

`/security-review` → 수정 → 재검
사

WORKFLOW 4

병렬 리팩토링

`/worktree-start` → 리뷰 → 머지

지금까지의 재료(커맨드, 에이전트, 혹)를 실무 시나리오로 조합합니다. 이 장이 이 책의 본론입니다.

워크플로우 1: 새 기능 개발 (TDD)

<code>/plan</code> 사용자 알림 설정 기능 (계획 검토·승인)	← <code>planner</code> 가 구현 계획 수립
<code>/tdd</code> (RED → GREEN → REFACTOR 사이클)	← 테스트 먼저 작성 강제
<code>/code-review</code>	← 심각도 기반 리뷰
<code>/quick-commit</code>	← 컨벤션 커밋

포인트: `/plan` 없이 바로 시작하면 Claude는 "일단 코드부터" 쓰는 경향이 있습니다. 계획 승인 단계를 강제하는 것만으로 대형 재작업이 크게 줄어듭니다.

워크플로우 2: 버그 수정

- 버그 재현 테스트를 먼저 작성하게 한다 ("이 버그를 재현하는 실패 테스트부터")
- 원인 분석을 시키되, 수정 전에 가설을 설명하게 한다
- 최소 수정으로 테스트를 통과시킨다
- `/verify-loop` 로 전체 테스트·린트·빌드 통과 확인

"버그 고쳐줘" 한 줄과 위 절차의 차이: 전자는 증상만 가리는 수정이 자주 나오고, 후자는 재발 방지 테스트가 남습니다.

워크플로우 3: 보안 감사

`/security-review` ← OWASP 기준 자동 점검
(`security-reviewer` 에이전트가 심각도별 리포트 생성)
발견 항목별로 수정 → 재검사 반복

워크플로우 4: 대규모 리팩토링 (병렬)

```
/worktree-start refactor-auth    ← 격리된 worktree 생성
(메인 작업과 병렬로 리팩토링 진행)
/code-review → 머지
/worktree-cleanup                ← 정리
```

git worktree를 쓰면 Claude 세션 여러 개를 서로 다른 브랜치에서 동시에 돌릴 수 있습니다. "리팩토링하는 동안 기능 개발이 멈추는" 문제가 사라집니다.

9장. 컨텍스트와 비용 관리 — 오래 달리기 위한 기술

Claude Code를 실무에 쓰다 보면 부딪히는 두 벽이 컨텍스트 한계와 토큰 비용입니다.

컨텍스트 관리 원칙

- 한 세션 = 한 작업**: 여러 작업을 한 세션에 섞으면 컨텍스트가 오염되어 품질이 떨어집니다. 작업이 끝나면 `/clear` 로 새로 시작하세요.
- `/compact` 는 마일스톤에서**: 긴 작업 중간중간 `/compact` 로 대화를 요약하면 컨텍스트를 절약하면서 맥락을 유지합니다. 자동 압축이 뜨기 전에 능동적으로.
- 탐색은 서브에이전트로**: "이 코드베이스에서 X를 찾아줘" 같은 탐색 작업은 서브에이전트에 위임하면 수백 페이지를 읽은 흔적이 메인 컨텍스트에 남지 않습니다.
- CLAUDE.md 다이어트**: 2장에서 말했듯 CLAUDE.md는 모든 세션에 로드됩니다. 여기가 비대하면 매 세션이 무겁습니다.

비용 절약 전략

- 기계적 작업(포매팅, 단순 변환, 문서 갱신)은 하위 모델로 충분합니다. 에이전트 정의의 `model` 필드로 역할별 모델을 고정하세요.
- 같은 질문을 반복하지 말고 CLAUDE.md나 memory MCP에 기록하세요. "기억 비용"은 한 번이지만 "재설정 비용"은 매번입니다.
- 대형 리팩토링 전에 `/plan` 으로 범위를 좁히는 것 자체가 최대의 비용 절감입니다.

10장. 트러블슈팅 FAQ — 자주 부딪히는 문제들

Q. Claude가 CLAUDE.md 규칙을 무시해요. 규칙이 너무 많거나 모호할 가능성이 높습니다. 규칙을 명령형 한 문장으로 줄이고, 가장 중요한 것을 파일 상단에 두세요. 그래도 반복되면 해당 규칙을 흑으로 승격해 구조적으로 막으세요. (규칙은 어길 수 있지만 흑은 못 어깁니다)

Q. 매번 권한 확인 버튼 누르는 게 지쳐요. 3장의 `permissions.allow` 목록에 안전한 명령을 추가하세요. 단, `Bash(*)` 같은 전체 허용은 금물입니다.

Q. 긴 작업을 시키면 중간에 방향이 틀어져요. 작업을 시작하기 전 `/plan` 으로 계획을 문서화하고, 중간마다 "계획 대비 현재 상태 점검"을 요청하세요. 설정팩의 `/checkpoint` 커맨드가 이 용도입니다.

Q. 다른 팀원과 설정을 공유하려면? `.claude/` 폴더(commands, agents, settings.json)와 `CLAUDE.md`, `.mcp.json` 을 git에 커밋하면 됩니다. 개인 API 키와 취향 설정만 `settings.local.json` 으로 분리.

Q. Claude가 임의로 위험한 명령을 실행할까 무서워요. 기본값은 모든 수정·실행에 승인이 필요하므로 안전합니다. 더 조이라면: deny 권한(3장) + 가드 혹(6장) + git 브랜치 보호를 조합하세요. 이 3중 구조면 자율 모드로 돌려도 사고 범위가 브랜치 하나로 제한됩니다.

Q. 설정팩은 어떻게 설치하나요? 부록을 참고하세요. 설치 스크립트가 `~/.claude` 에 심볼릭 링크를 만들어 모든 프로젝트에서 커맨드·에이전트를 공유하는 구조입니다.

부록. 동봉 설정팩(claude-forge) 설치 가이드

이 책의 모든 예제가 들어 있는 설정팩은 오픈소스(MIT)로 제공됩니다.

설치 (macOS / Linux / WSL)

```
git clone <설정팩 저장소 URL>
cd claude-forge
./install.sh
```

설치 스크립트는 `~/.claude/` 아래에 commands, agents, skills, hooks, rules의 심볼릭 링크를 생성합니다. 원본을 git으로 관리하므로 `git pull` 만으로 전체 설정이 업데이트됩니다.

구성 요소 요약

폴더	내용	개수
commands/	슬래시 커맨드	35종
agents/	서브에이전트	11종
hooks/	자동화 혹 스크립트	15종
rules/	규칙 파일 (코딩·git·보안·테스트)	8종
skills/	스킬 패키지	15종

커스터마이징 원칙

- 원본 파일을 직접 수정하지 말고 `-custom` 접미사 폴더에 복사 후 수정 (업데이트 시 충돌 방지)
- 프로젝트 고유 설정은 프로젝트의 `.claude/` 에, 범용 설정은 전역에
- 안 쓰는 커맨드는 과감히 삭제 — 커맨드 목록도 컨텍스트를 차지합니다

맷으며 — 도구가 아니라 시스템을 만드세요

Claude Code를 잘 쓰는 사람과 못 쓰는 사람의 차이는 프롬프트 실력이 아닙니다. **반복되는 것을 시스템(커맨드·에이전트·훅)으로 승격시키는 습관**의 차이입니다.

- 같은 지시를 두 번 타이핑했다면 → 커맨드로
- 같은 역할을 두 번 시켰다면 → 에이전트로
- 같은 실수를 두 번 봤다면 → 훅으로

이 책의 설정팩은 그 시스템의 출발점입니다. 그대로 쓰기보다, 여러분의 프로젝트에서 반복되는 것들을 하나씩 엮어가며 **여러분만의 포지(대장간)**로 키워가시길 바랍니다.